

Fault-Onset Labeling Bounds Reported Accuracy in the Tennessee Eastman Benchmark: A Leakage-Controlled Study of Six Classical Classifiers

Abstract

Fault diagnosis in the Tennessee Eastman Process (TEP) is a standard benchmark for data-driven process monitoring, yet published results are difficult to compare. We present a leakage-controlled benchmark of six classical classifiers: logistic regression, decision tree, random forest, histogram-based gradient boosting, linear support vector machine and XGBoost. Data were partitioned by complete simulation runs, standardization was fitted on training data only, hyperparameters were selected on a separate validation partition, variability was measured over five seeds, and the frozen model was evaluated once on the complete official test partition of 10.08 million observations. XGBoost ranked first in repeated validation (macro-F1 0.7683 ± 0.0017) and reached a test macro-F1 of 0.7200, accuracy of 0.6754 and MCC of 0.6621 under the labels distributed with the dataset.

These figures are bounded by the benchmark itself. Each fault is injected 160 samples into every 960-sample test run, so literal labeling caps the recall of every fault class at 0.8333; faults 6 and 7 attain 0.8333 and 0.8332, the ceiling itself. Rescoring the same frozen model over the post-onset segment, without refitting, raises macro-F1 to 0.8054, accuracy to 0.7949 and MCC to 0.7852, and the apparent validation-to-test generalization gap of +0.0483 becomes -0.0125 under a symmetric comparison. Across seven classifiers the relative accuracy gain lies between 0.142 and 0.150 against a structural bound of 0.1587, so the effect is a property of the labeling convention rather than of any estimator. Normal operation and faults 3, 9 and 15 remain mutually confounded after the correction, consistent with their known weak observability.

Keywords: Tennessee Eastman Process, fault diagnosis, machine learning, process monitoring, reproducible benchmark, fault-onset labeling

31 1. Introduction

32 Industrial processes operate through interacting physical, chemical, and
33 control mechanisms. A local disturbance can propagate through feedback
34 loops, alter several measured variables, and eventually compromise product
35 quality, equipment integrity, safety, or environmental performance. Fault
36 detection and diagnosis (FDD) therefore constitute central elements of mod-
37 ern process monitoring [1, 2, 3]. Detection determines whether the process
38 has departed from acceptable operation, while diagnosis seeks to identify the
39 underlying condition so that operators or automated systems can respond
40 appropriately.

41 Model-based diagnosis can be effective when reliable first-principles mod-
42 els are available. However, plant-wide systems are frequently nonlinear, high-
43 dimensional, stochastic, and only partially observed. These properties have
44 motivated data-driven monitoring, in which statistical or machine-learning
45 models infer diagnostic boundaries from historical measurements [4, 2]. Clas-
46 sical multivariate methods such as principal-component analysis (PCA), par-
47 tial least squares (PLS), and Fisher discriminant analysis established impor-
48 tant foundations for process monitoring [5]. More recent studies increasingly
49 employ tree ensembles, support vector machines, neural networks, and se-
50 quence models to represent nonlinear relationships and dynamic behavior.

51 The Tennessee Eastman Process (TEP), originally introduced by Downs
52 and Vogel [6], remains one of the most widely used benchmarks in this field.
53 It represents a reactor–separator–recycle chemical process with 41 measured
54 variables, 11 manipulated variables, and a diverse set of disturbances. Its
55 continued value arises from the coexistence of relatively distinctive faults
56 and difficult conditions whose trajectories overlap normal operation. The
57 expanded simulation data published by Rieth et al. [7] provide hundreds
58 of independent runs per condition and non-overlapping random seeds for
59 development and testing, enabling large-scale and reproducible evaluation.

60 Despite extensive use of the TEP, reported results are often difficult to
61 compare. Studies may split temporally adjacent observations rather than
62 complete runs, tune models on the test set, report only accuracy, or omit
63 class-level behavior. Treating autocorrelated observations as independent
64 experimental replicates can produce optimistic estimates, while a high global
65 score can hide failures in specific operating conditions. These issues are es-
66 pecially relevant when the normal state is confused with a fault, because false
67 alarms can reduce operator confidence and generate unnecessary interven-

tions.

This study addresses these limitations through a leakage-controlled benchmark of six classical supervised classifiers: logistic regression, decision tree, random forest, histogram-based gradient boosting, linear support vector machine (SVM), and XGBoost. The study asks five questions:

1. Which classical model provides the strongest macro-averaged diagnostic performance under repeated validation?
2. Does the selected model maintain its performance on a complete, independent official test partition?
3. Which operating conditions account for the remaining diagnostic errors?
4. How much of the reported error is attributable to the fault-onset labeling convention rather than to the classifier?
5. Which model offers the best compromise between diagnostic performance and computational cost?

The main contribution is an auditable protocol that separates model selection from final testing, preserves complete simulation runs, quantifies variability over five seeds, reports confidence intervals, and examines precision, recall, F1, and confusion patterns for all 21 classes. Rather than presenting a single headline score, the analysis identifies a compact ambiguity group that defines the practical limit of the selected classifier.

A second contribution is methodological and, in our view, of wider consequence for the TEP literature. The expanded dataset injects each fault after a fixed warm-up interval, so a fraction of every faulty run is nominally normal yet carries the fault label. This fraction differs between development runs (4%) and official test runs (16.7%). We quantify how much of the reported error and of the apparent generalization gap is produced by this convention alone, and we report every headline metric under both the literal labeling and a post-onset labeling. To our knowledge this decomposition is not routinely reported, which may account for part of the dispersion among published TEP results.

2. Related Work

2.1. Data-driven process monitoring

Data-driven FDD spans latent-variable modeling, classification, clustering, change detection, and temporal modeling. The review by Venkatasubramanian et al. [1] established a broad taxonomy covering quantitative models,

104 qualitative models, and process-history-based methods. Chiang et al. [5, 4]
 105 showed how PCA, discriminant PLS, and Fisher discriminant analysis could
 106 support chemical-process diagnosis. Later reviews emphasized that industrial
 107 data may be nonlinear, dynamic, nonstationary, multimodal, and autocorre-
 108 lated [2, 3]. These properties motivate flexible nonlinear classifiers, but also
 109 demand careful validation.

110 Multivariate statistical monitoring of the TEP has a long history. Raich
 111 and Çinar combined statistical process monitoring with disturbance diagnosis
 112 in multivariable continuous processes [25], and multiscale extensions using
 113 wavelet decompositions were introduced to separate phenomena occurring
 114 at different time scales [27]. A broader survey of plant-wide disturbance
 115 detection is given by Thornhill and Horch [30].

116 A substantial line of work addresses the serial correlation of process data
 117 directly. Dynamic PCA augments the data matrix with time-lagged variables
 118 to capture time-varying structure [40]; canonical variate analysis models the
 119 dynamics explicitly and was compared against PCA and DPCA on the TEP
 120 simulator by Russell et al. [41], later extended with kernel density estimation
 121 for nonlinear dynamics [42]. These methods are relevant here for a reason
 122 beyond their performance: they treat the temporal structure of a run as
 123 the object of analysis, whereas an observation-level classifier treats each row
 124 independently. The labeling question examined in Section 3.2 arises precisely
 125 at that boundary.

126 2.2. Classical machine learning for TEP diagnosis

127 The families evaluated here are standard: classification and regression
 128 trees [8], random forests [9], gradient boosting [10] and its regularized histogram-
 129 based formulation in XGBoost [11], together with multinomial logistic regres-
 130 sion and margin-based linear classification [12] as controls that test whether
 131 approximately linear decision surfaces suffice. All implementations in this
 132 study were based on scikit-learn [13], except XGBoost, which used its refer-
 133 ence implementation [11].

134 Support vector machines have been applied to the TEP in several forms.
 135 Chiang et al. combined Fisher discriminant analysis with support vector
 136 machines for fault diagnosis [28]; Kulkarni et al. incorporated process knowl-
 137 edge into the kernel to detect TEP faults [29]; and kernel extensions of PCA
 138 and independent component analysis were compared against SVM on the
 139 same benchmark [31]. These studies use kernels that the present work could
 140 not afford at the sample size considered here, which is the reason the linear

141 surrogate of Section 3.4 should be read as a lower bound rather than as the
142 best attainable margin classifier.

143 Deep architectures have been applied extensively to the same benchmark.
144 Chadha and Schwung compared feedforward and convolutional networks for
145 TEP fault detection [36]; hybrid designs combining support vector machines
146 with latent-variable preprocessing [39], hierarchical LSTM structures [37],
147 and metaheuristically optimized deep models [38] have all reported strong
148 results. Most directly comparable to the present study, Lomov et al. [35]
149 evaluated recurrent, attention-based and temporal convolutional architec-
150 tures on the same expanded TEP dataset used here. The present work does
151 not propose a competing architecture; it examines a property of the evalua-
152 tion protocol that applies to all of them.

153 *2.3. Evaluation protocols and leakage*

154 Concern about evaluation protocol is not specific to process monitoring.
155 In a survey across disciplines that have adopted machine learning, Kapoor
156 and Narayanan identified leakage in seventeen fields, affecting at least 294
157 publications and in several cases producing conclusions that did not survive
158 correction [34]. Their taxonomy distinguishes eight mechanisms, two of which
159 bear directly on the present benchmark: temporal leakage, where information
160 from the future informs a prediction about the past, and non-independence
161 between training and evaluation samples, where records that share a common
162 origin are distributed across partitions and treated as independent replicates.

163 Both are latent in observation-level TEP classification. Consecutive sam-
164 ples within a simulation run are strongly autocorrelated, so a split performed
165 at the observation level places near-duplicate records on both sides of the par-
166 tition. The run-level protocol of Section 3.3 is designed to avoid this. The
167 labeling question analyzed in this paper is related but distinct: it concerns
168 not how records are divided, but whether the label attached to a record
169 describes the state of the plant at that instant.

170 *2.4. Evaluation beyond accuracy*

171 Accuracy can be misleading when performance varies substantially among
172 classes. Macro-F1 gives equal importance to every class by averaging class-
173 specific F1 scores, while weighted-F1 weights them by support. Balanced
174 accuracy averages class recalls. MCC summarizes agreement between pre-
175 dictions and true labels and remains informative for multiclass problems [14].
176 Confusion matrices are indispensable because they reveal whether errors are

Table 1: Treatment of the pre-onset window in observation-level studies on the Tennessee Eastman benchmark. Three studies on the same simulator adopt three distinct conventions: relabeling the pre-onset segment as normal, retaining it under the fault label, and not addressing it.

Study	Task	Methods	Pre-onset handling	Scored as fault
Yin et al. [32]	binary detection	SVM, PLS	stated, relabeled normal	800
Sun et al. [33]	detection + identif.	Bayesian RNN	stated, retained	960
Agarwal et al. [37]	21-class diagnosis	LSTM-SAE	not mentioned	960
This work	21-class diagnosis	six classical	stated, both protocols	960 / 800

diffuse or concentrated in specific class pairs. The present study therefore uses macro-F1 as the primary model-selection criterion and treats the other metrics as complementary evidence.

2.5. Reported treatment of the fault-onset window

The analysis reported in this paper is only consequential if the treatment of the onset window varies, or goes unstated, across published work on the same benchmark. We therefore examined the data-description and experimental-protocol sections of studies for which the full text was available, recording the diagnostic task, the stated treatment of the pre-onset segment, and the number of test samples scored against the fault label. The purpose is illustrative rather than exhaustive. Three studies on the same simulator adopt three different conventions, which is sufficient to establish that the choice is not standardized; Table 1 reports them.

The question applies to both distributions of the benchmark. In the original data of Downs and Vogel as distributed by Chiang et al. [6, 4], each condition is simulated for 24 hours in training and 48 hours in testing at a three-minute interval, giving 480 and 960 samples, with the fault injected after one hour and eight hours respectively. The pre-onset fractions are therefore 4.2% and 16.7%, essentially identical to those of the expanded dataset analyzed here. The artifact is thus a property of the benchmark design rather than of any particular redistribution of it, and the results of Section 4.5 bound what can be reported on either version.

Yin et al. [32] address the window directly. Their data description states that fault information emerges eight hours after start-up, so that the first 160 test samples appear normal while the remaining 800 are genuinely faulty, and their discussion of the classification output is explicit about the consequence: the target label is -1 over the first 160 samples and $+1$ from the 161st

204 onward. The pre-onset segment is thus relabeled as normal rather than
205 scored against the fault, and their fault-detection rate is computed over 800
206 samples.

207 Sun et al. [33] state the same structure – the simulator runs 160 samples
208 in the normal state before the disturbance is introduced, then continues for
209 800 further samples – but evaluate over all 960. Awareness of the injection
210 instant is therefore not sufficient by itself; what determines comparability is
211 what is done with the segment.

212 Agarwal et al. [37] describe their data as 1440 samples per class, of which
213 the first 480 are used for calibration and the remaining 960 for testing, and
214 identify the source distribution explicitly. The instant of fault injection is
215 not mentioned anywhere in the data description or the experimental proto-
216 col. The consequence is not incidental to that study’s argument: its stated
217 objective is to improve the diagnosis of the incipient faults 3, 9 and 15, to
218 which end pseudo-random binary signals are actively injected into the plant.
219 Under literal labeling, part of the difficulty that motivates such an interven-
220 tion is contributed by observations in which no fault is yet present.

221 The pattern across the three is not one of carelessness, and it admits
222 a structural explanation. In binary detection the pre-onset segment has a
223 natural destination: the normal class already exists in the problem, so rela-
224 beling those observations is the obvious treatment, and the false-alarm rate
225 is in fact defined over precisely that segment. The convention is therefore
226 built into the evaluation. In multiclass diagnosis the same move is no longer
227 neutral, because assigning pre-onset observations of every faulty run to class
228 0 alters the prior of the normal condition and, in the present data, would
229 add 1.6 million observations to a class that already competes with faults 3,
230 9 and 15. Faced with that, the simplest course is to label each run by its
231 fault number throughout, and the distinction disappears without ever being
232 posed as a choice. What is lost in the transition from detection to diagnosis
233 is not information about the data but a convention for handling it.

234 The practical consequence is that reported figures are not commensurable
235 across the three treatments. For a single frozen model the difference between
236 literal and post-onset labeling is 0.0854 in macro-F1 and 0.1231 in MCC (Sec-
237 tion 4.5), larger than the margins that typically separate competing methods
238 on this benchmark. We do not claim a systematic review here; the point is
239 that three conventions coexist, that the choice is seldom identified as such,
240 and that it should be reported alongside the unit of partitioning.

241 3. Materials and Methods

242 3.1. Tennessee Eastman Process data

243 The TEP simulates a plant-wide reactor–separator–recycle system and
 244 includes 41 process measurements (XMEAS) and 11 manipulated variables
 245 (XMV), totaling 52 predictors [6]. The original problem statement specifies
 246 the plant but not a control structure; the simulations distributed as bench-
 247 mark data use the plant-wide control scheme of Lyman and Georgakis [26],
 248 so the recorded trajectories reflect both the process dynamics and the closed
 249 loops acting on them. The classification task contained 21 labels: class 0 rep-
 250 resented fault-free operation and classes 1–20 represented the predefined fault
 251 conditions. The CSV distribution used in this work is a format conversion
 252 of the expanded TEP simulation data of Rieth et al. [7, 15].

253 The four source files contained 15,330,000 observations and 55 columns:
 254 the 52 process variables plus the identifiers `faultNumber`, `simulationRun`,
 255 and `sample`. Development runs contained 500 observations each. Official
 256 test runs contained 960 observations each. Every class had 500 official test
 257 runs, yielding

$$21 \times 500 \times 960 = 10,080,000 \quad (1)$$

258 test observations. Each class therefore contributed 480,000 observations
 259 to the final test.

260 3.2. Fault-onset windows and labeling protocols

261 The expanded dataset is generated by simulating each condition for 25
 262 hours (development) or 48 hours (official test) with a sampling interval of
 263 three minutes, and by injecting the fault after a fixed warm-up interval: one
 264 hour in faulty development runs and eight hours in faulty test runs [7, 16, 17].
 265 In sample indices, the fault is active from sample 21 in development runs and
 266 from sample 161 in official test runs. Fault-free runs contain no injection.

267 Consequently, the labels distributed with the dataset are only condition-
 268 ally correct. In a faulty development run, samples 1–20 (4.0% of the run)
 269 describe nominal operation while carrying a fault label; in a faulty official
 270 test run, samples 1–160 (16.7% of the run) do so. Two consequences follow
 271 directly and are, to our knowledge, seldom made explicit.

272 First, under an observation-level protocol with literal labels, the recall of
 273 every fault class is bounded above by

$$R_c^{\max} = \frac{960 - 160}{960} = 0.8333, \quad c = 1, \dots, 20, \quad (2)$$

274 whenever the classifier assigns pre-onset observations to some class other
 275 than c . No such ceiling applies to class 0, whose test runs contain no injection.
 276 Second, because the mislabeled fraction differs between development (4.0%)
 277 and test (16.7%), a metric computed on the two partitions is not measuring
 278 the same quantity, and part of any observed validation-to-test degradation
 279 is attributable to this asymmetry rather than to generalization.

280 We therefore report all headline metrics under two labeling protocols:

- 281 • **Literal labeling (L)**: every observation retains the label distributed
 282 with the dataset. This is the protocol implicitly adopted by most pub-
 283 lished observation-level TEP results and is retained here for compara-
 284 bility.
- 285 • **Post-onset labeling (P)**: pre-onset observations of faulty runs are
 286 excluded *at evaluation time*, so that each scored observation carries a
 287 label consistent with the physical state of the plant. The model is not
 288 refitted: protocol P rescores the predictions of the same frozen model
 289 that was fitted under protocol L, including its 4% of contaminated
 290 training observations. The comparison L versus P therefore isolates
 291 the labeling convention alone, and the reported post-onset figures are
 292 conservative, since a model additionally trained without the contami-
 293 nated observations could only do better.

294 A third variant, in which pre-onset observations are excluded from fitting
 295 as well, was not evaluated here; it would confound the labeling effect with a
 296 change in the training distribution and is left for future work.

297 Protocol L is used for model selection, so that the frozen-model decision
 298 is unaffected by the present analysis. Protocol P is reported alongside it in
 299 Section 4.5. A third option, relabeling pre-onset observations as class 0, was
 300 not adopted because it alters the class prior of the normal condition and
 301 would confound the analysis of the ambiguity group.

302 3.3. *Leakage-controlled partitioning and preprocessing*

303 The original development runs were divided at the run level into training
 304 and validation partitions using a fixed random seed and a stratified 80/20
 305 split within each source and class. This produced 8,400 training runs (400

306 per class), 2,100 validation runs (100 per class), and 10,500 official test runs
 307 (500 per class). No run was assigned to more than one partition.

308 All 52 predictors were standardized using

$$z_{ij} = \frac{x_{ij} - \mu_j^{(\text{train})}}{\sigma_j^{(\text{train})}}, \quad (3)$$

309 where $\mu_j^{(\text{train})}$ and $\sigma_j^{(\text{train})}$ were estimated exclusively from the 4.2 million
 310 training observations. The fitted transformation was then applied unchanged
 311 to validation and test data. This procedure prevented information from the
 312 validation or test partitions from influencing preprocessing.

313 3.4. Candidate classifiers

314 Six model families were compared. Hyperparameter tuning evaluated
 315 three candidates per family, producing 18 configurations. The selected con-
 316 figurations were subsequently frozen:

- 317 • **Logistic regression:** $C = 1.0$, maximum 1,000 iterations;
- 318 • **Decision tree:** unrestricted depth and minimum leaf size 1;
- 319 • **Random forest:** 400 trees, unrestricted depth, 50% of features con-
 320 sidered per split, and minimum leaf size 1;
- 321 • **HistGradientBoosting:** learning rate 0.05, 350 boosting iterations,
 322 and at most 31 leaf nodes;
- 323 • **Linear SVM:** stochastic-gradient hinge-loss classifier with $\alpha = 10^{-5}$,
 324 maximum 2,000 iterations, and tolerance 10^{-3} ;
- 325 • **XGBoost:** 250 trees, maximum depth 6, learning rate 0.1, row sub-
 326 sampling 0.8, column subsampling 0.8, multiclass probabilistic objec-
 327 tive, and histogram tree construction.

328 The tuning stage ranked configurations by macro-F1 and used MCC as a
 329 secondary criterion. The official test partition was not accepted as an input
 330 by either the tuning or repeated-validation programs.

331 The linear SVM was implemented as a stochastic-gradient surrogate rather
 332 than as an exact margin solver, because quadratic-programming solvers were
 333 not tractable at this sample size. Its score should therefore be read as a lower

bound on linear-margin performance, and the evidence for nonlinearity reported in Section 5.1 rests primarily on the multinomial logistic regression, which was optimized to convergence. Kernel SVMs were not evaluated for the same computational reason.

3.5. Repeated validation

After selecting the best configuration for each model family, variability was measured across seeds 2026–2030. For each seed, 40 complete runs per class were sampled from training and independently from validation. Thus, every repetition used 420,000 training and 420,000 validation observations while retaining the full 500-observation trajectories of all selected runs.

For each metric, the mean, sample standard deviation, and 95% confidence interval were calculated across the five repetitions. With $n = 5$, the interval was

$$\bar{x} \pm t_{0.975,4} \frac{s}{\sqrt{5}}. \quad (4)$$

Because only five seeds were used, these intervals quantify observed seed-to-seed variability but should not be interpreted as definitive evidence of pairwise statistical superiority. In particular, overlapping intervals for the two leading models were interpreted conservatively.

In addition to the interval estimates, the six model families were compared with the Friedman rank test over the five seeds treated as blocks, followed by the Nemenyi post-hoc procedure [19]. Pairwise comparisons used the Wilcoxon signed-rank test with Holm correction. The critical difference of the Nemenyi procedure was computed as $CD = q_\alpha \sqrt{k(k+1)/6N}$ with $k = 6$ models and $N = 5$ blocks. Results are reported in Section 4.1. With five blocks these procedures have low power by construction, and they are reported to document that the prespecified ranking rule was not substituted for an inferential check, not to establish pairwise superiority.

Because a single frozen evaluation cannot express sampling uncertainty, the official-test metrics were additionally accompanied by 95% intervals obtained by resampling complete runs with replacement (1,000 bootstrap replicates over the 10,500 test runs). The run, not the observation, is the resampling unit, consistent with the argument that temporally adjacent observations are not independent replicates.

366 3.6. Frozen final evaluation

367 XGBoost candidate 1 was selected because it achieved the highest repeated-
368 validation macro-F1. The final seed was fixed at 2026 before test evaluation.
369 The final development set combined 40 complete runs per class from train-
370 ing and 40 from validation, totaling 840,000 observations. The frozen model
371 was then evaluated once on all 10,080,000 official test observations, without
372 subsampling or post-test adjustment.

373 Two earlier executions aborted during structural validation, before any
374 model fitting or metric computation, because of an incorrect run-length as-
375 sertion; only that assertion was changed, and the full incident log is available
376 in the repository.

377 The choice of 840,000 development observations, rather than the full
378 5,250,000 available, was imposed by the computational budget. To bound the
379 effect of that choice, the frozen configuration was refitted on $\{10, 20, 40, 80, 160\}$
380 complete runs per class and scored on the full validation partition, sampling
381 at the run level throughout. Results are reported in Section 4.2.

382 3.7. Performance measures and reproducibility

383 The reported measures were accuracy, balanced accuracy, precision, re-
384 call, class-specific F1, macro-F1, weighted-F1, MCC, confusion matrix, train-
385 ing time, inference time, inference time per observation, and serialized model
386 size. Zero divisions in class-specific measures were assigned zero. Data and
387 output integrity were monitored with SHA-256 hashes and manifests. The
388 final execution used Python 3.12.13, scikit-learn 1.9.0, and XGBoost 3.4.0 on
389 Linux.

390 4. Results

391 4.1. Model selection under repeated validation

392 Table 2 summarizes the five-seed validation results. XGBoost ranked first
393 with macro-F1 of 0.7683 ± 0.0017 and MCC of 0.7426 ± 0.0018 . HistGradi-
394 entBoosting was a close second, with macro-F1 of 0.7643 ± 0.0034 . Random
395 forest achieved 0.7398, followed by decision tree at 0.6714. The two linear
396 baselines performed substantially worse: logistic regression achieved 0.4685
397 and linear SVM achieved 0.4438.

398 The Friedman test over the five seeds rejected the hypothesis of equal
399 performance among the six families ($\chi^2_5 = 24.54$, $p = 1.71 \times 10^{-4}$), with a

Table 2: Repeated-validation performance over five seeds. Macro-F1 was the prespecified primary selection criterion.

Rank	Model	Macro-F1 (mean $\pm$ SD)	95% CI	MCC (mean $\pm$ SD)
1	XGBoost	0.7683 ± 0.0017	[0.7661, 0.7705]	0.7426 ± 0.0018
2	HistGradientBoosting	0.7643 ± 0.0034	[0.7601, 0.7686]	0.7391 ± 0.0045
3	Random Forest	0.7398 ± 0.0021	[0.7372, 0.7424]	0.7159 ± 0.0021
4	Decision Tree	0.6714 ± 0.0013	[0.6699, 0.6730]	0.6537 ± 0.0015
5	Logistic Regression	0.4685 ± 0.0074	[0.4592, 0.4777]	0.4567 ± 0.0073
6	Linear SVM	0.4438 ± 0.0099	[0.4314, 0.4561]	0.4377 ± 0.0109

400 Kendall coefficient of concordance of $W = 0.982$. The ordering is thus almost
401 perfectly consistent from seed to seed: XGBoost obtained the best macro-F1
402 in four of the five repetitions, and the mean ranks were 1.2 (XGBoost), 1.8
403 (HistGradientBoosting), 3.0 (random forest), 4.0 (decision tree), 5.0 (logistic
404 regression) and 6.0 (linear SVM).

405 Post-hoc resolution is nevertheless limited. With $N = 5$ blocks the Ne-
406 menyí critical difference is 3.372 mean ranks, so only three of the fifteen
407 pairwise comparisons are individually significant, all of them between a tree
408 ensemble and a linear baseline. The separation between XGBoost and Hist-
409 GradientBoosting is 0.6 mean ranks, far below the critical difference. The
410 Wilcoxon signed-rank test cannot help here either: with five paired observa-
411 tions its smallest attainable two-sided p -value is 0.0625, so no comparison can
412 reach $\alpha = 0.05$ even under a perfect win record, and after Holm correction
413 none does.

414 These two facts are complementary rather than contradictory. The high
415 concordance shows that the ranking is not an artifact of seed choice; the
416 wide critical difference shows that five repetitions cannot certify small dif-
417 ferences. Both support the conservative reading adopted here: the ordering
418 is reproducible, the gap between the two leading models is not resolvable at
419 this sample size, and additional seeds would be required to settle it.

420 The confidence intervals of XGBoost and HistGradientBoosting over-
421 lapped. Accordingly, XGBoost was selected by the prespecified macro-F1
422 rule, but the result is not presented as proof of a statistically significant
423 difference between the two models. The large separation between nonlin-
424 ear ensembles and linear baselines indicates that nonlinear interactions are
425 central to this diagnostic task.

Table 3: Validation macro-F1 as a function of the training set size, in complete runs per class. Δ is the gain over the preceding row, that is, the effect of doubling the data.

Runs/class	Observations	F1 (L)	Δ	F1 (P)	Δ	Fit (s)
10	105,000	0.7538	—	0.7780	—	59.0
20	210,000	0.7591	0.0052	0.7847	0.0067	65.2
40	420,000	0.7672	0.0082	0.7930	0.0083	111.6
80	840,000	0.7756	0.0083	0.7988	0.0058	175.4
160	1,680,000	0.7806	0.0051	0.8016	0.0028	357.4

4.2. Sensitivity to the size of the development set

Table 3 reports macro-F1 on the complete validation partition as a function of the number of complete runs per class used for fitting, under both labeling protocols. All fits used the frozen configuration and 24 threads.

Two observations follow. First, the procedure reproduces the repeated-validation result: at 40 runs per class it yields a macro-F1 of 0.7672 against the 0.7683 ± 0.0017 of Table 2, a difference of 0.0011, which confirms that the curve is measured under the protocol of Section 3.5 rather than under a variant of it.

Second, the returns diminish. Doubling the data from 80 to 160 runs per class adds 0.0051 in macro-F1 under literal labeling and 0.0028 under post-onset labeling; quadrupling from the 40 runs per class used in the frozen fit adds 0.0134 and 0.0086 respectively. The frozen model is therefore not at the asymptote, and the figures reported in Section 4.3 are conservative in that sense, but the residual headroom is of the same order as the separation between the two leading model families and does not alter the ranking.

The comparison that matters for this paper is between that headroom and the effect under study. Quadrupling the training data from the size actually used gains 0.0086 in post-onset macro-F1; changing the labeling convention on a single fixed model changes macro-F1 by 0.0854, an order of magnitude more. Whatever remains to be gained from more data, it is small beside what is gained or lost by how the pre-onset window is scored.

4.3. Official test performance

After the protocol was frozen, the selected XGBoost model was fitted to 840,000 development observations and evaluated on the complete official test set. Table 4 presents the final metrics under literal labeling. Accuracy and

452 balanced accuracy were both 0.6754, macro-F1 and weighted-F1 were both
 453 0.7200, and MCC was 0.6621. The equality of accuracy and balanced accu-
 454 racy, and of macro-F1 and weighted-F1, follows from the exactly balanced
 455 class supports, and holds only under literal labeling; under the post-onset
 456 protocol of Section 4.5 the fault classes retain 400,000 observations while the
 457 fault-free class retains 480,000, so the two pairs of metrics separate. Macro-
 458 F1 exceeds accuracy because accuracy equals macro-recall under balanced
 459 supports, while precision exceeds recall in most classes; the harmonic mean
 460 of a high precision and a moderate recall is larger than the recall alone. The
 461 gap between macro-F1 and accuracy is therefore itself a signature of the
 462 systematic under-recall documented in Section 4.5.

Table 4: Frozen XGBoost performance on the official test set.

Metric	Value
Accuracy	0.6754
Balanced accuracy	0.6754
Macro-F1	0.7200
Weighted-F1	0.7200
MCC	0.6621
Training time (s)	170.34
Inference time (s)	56.57
Inference (ms/observation)	0.005612
Development observations	840,000
Test observations	10,080,000

463 Relative to repeated validation, macro-F1 decreased by 0.0483 and MCC
 464 by 0.0805. Three mechanisms could account for this difference: independent
 465 random states in the test simulations, the longer duration of test runs, and
 466 the larger fraction of mislabeled pre-onset observations. The third mechanism
 467 is both the largest and the only one that is purely a labeling artifact. Under
 468 literal labeling the attainable recall per fault class falls from $480/500 = 0.96$
 469 in development runs to $800/960 = 0.8333$ in test runs, a ratio of 0.868, which
 470 is of the same order as the observed degradation. Section 4.5 shows that the
 471 gap does not merely shrink under post-onset labeling but changes sign: the
 472 post-onset test macro-F1 of 0.8054 exceeds the literal-label validation figure
 473 of 0.7683 by 0.0371.

474 That comparison alone would not be symmetric, because the validation
 475 figure was itself computed under literal labels and carries its own 4% pre-

onset contamination. The validation partition was therefore rescored under both protocols as well. Applying the selected XGBoost configuration to all 1,050,000 validation observations gave a literal-label macro-F1 of 0.7706, which agrees with the five-seed repeated-validation figure of 0.7683 to within 0.0023 and confirms that this partition is a faithful reference. Excluding the 40,000 pre-onset observations, 3.81% of the partition, raised macro-F1 to 0.7953, accuracy from 0.7556 to 0.7843, and MCC from 0.7440 to 0.7739.

The correction on the validation partition is thus +0.0246 in macro-F1, against +0.0854 on the test partition, a ratio of 0.29 that tracks the ratio of contaminated fractions (3.81% against 15.87%). Transferring the measured validation correction to the repeated-validation reference gives a post-onset validation macro-F1 of approximately 0.7929, against a post-onset test macro-F1 of 0.8054. Table 5 summarizes the decomposition.

Table 5: Validation-to-test difference in macro-F1 under both labeling protocols. The gap present under literal labeling does not survive a symmetric comparison.

Protocol	Validation	Test	Difference
Literal (L)	0.7683	0.7200	+0.0483
Post-onset (P)	0.7929	0.8054	−0.0125

Under a symmetric comparison the gap is not merely reduced but reversed, and its magnitude falls to roughly a quarter of the literal-label value, within the range of the seed-to-seed variability of Table 2. We therefore conclude that the reported generalization gap is an artifact of the labeling convention rather than evidence of degradation on unseen trajectories, and that the literal-label figure of 0.7200 understates the discriminative capacity of the frozen model by 0.0854 in macro-F1.

Two caveats attach to this conclusion. The validation rescaling used the tuned configuration fitted to the full training partition, not the five per-seed refits of the repeated-validation procedure, so the transferred correction is an estimate of the effect rather than an exact recomputation; the close agreement between 0.7706 and 0.7683 bounds the error of that substitution. Furthermore, the pre-onset destinations differ between partitions: 61.1% of pre-onset validation observations were assigned to the group $\{0, 3, 9, 15\}$ against 92.0% in the test partition, which is consistent with the development segment being only 20 samples long and overlapping the simulation transient.

Table 6: Class-wise results for the frozen XGBoost model on the official test set. Every class had support of 480,000 observations.

Class	Precision	Recall	F1	Class	Precision	Recall	F1
0	0.1481	0.2045	0.1718	11	0.8447	0.7095	0.7712
1	0.9894	0.8243	0.8993	12	0.9271	0.7669	0.8395
2	0.9896	0.8209	0.8974	13	0.9828	0.7444	0.8471
3	0.1636	0.4218	0.2357	14	0.9889	0.8187	0.8958
4	0.9477	0.8222	0.8805	15	0.1465	0.1978	0.1683
5	0.9879	0.8234	0.8982	16	0.9756	0.6681	0.7931
6	0.9973	0.8333	0.9080	17	0.9624	0.7809	0.8622
7	0.9964	0.8332	0.9075	18	0.9367	0.7760	0.8488
8	0.9665	0.7779	0.8620	19	0.7701	0.7559	0.7630
9	0.1328	0.2126	0.1635	20	0.8391	0.7029	0.7650
10	0.8066	0.6885	0.7429				

505 4.4. Class-specific performance

506 Table 6 demonstrates marked heterogeneity. Seventeen classes achieved
507 F1 scores from 0.7429 to 0.9080. Classes 6 and 7 were strongest, with F1 of
508 0.9080 and 0.9075, respectively. Classes 1, 2, 5, and 14 also approached or
509 exceeded 0.8958.

510 Four classes were persistently difficult: class 9 (F1=0.1635), class 15
511 (0.1683), normal operation (class 0; 0.1718), and class 3 (0.2357). Class
512 3 had the highest recall within this group (0.4218) but very low precision
513 (0.1636), indicating systematic overprediction.

514 A pattern in the recall column deserves emphasis. Classes 6 and 7 reached
515 recalls of 0.8333 and 0.8332, which coincide with the structural ceiling of
516 Eq. (2) to within one part in ten thousand. Classes 1, 2, 4, 5 and 14 lie
517 between 0.8187 and 0.8243, that is, at 98–99% of that ceiling. The frozen
518 model is therefore essentially saturated on these classes once the pre-onset
519 segment is removed, and their reported F1 values of roughly 0.88–0.91 are
520 limited by the labeling convention rather than by the classifier.

521 4.5. Effect of the fault-onset window

522 Table 7 compares the frozen model under the two labeling protocols of
523 Section 3.2. No refitting, retuning or reselection was performed; the same
524 frozen model and the same predictions are scored under two label conven-
525 tions.

Table 7: Frozen XGBoost under literal (L) and post-onset (P) labeling on the official test partition. Intervals are 95% run-level bootstrap intervals over 1,000 replicates.

Metric	Literal (L)	Post-onset (P)
Accuracy	0.6754 [0.6716, 0.6797]	0.7949 [0.7898, 0.8006]
Balanced accuracy	0.6754 [0.6748, 0.6760]	0.8006 [0.7998, 0.8013]
Macro-F1	0.7200 [0.7192, 0.7208]	0.8054 [0.8044, 0.8063]
MCC	0.6621 [0.6579, 0.6666]	0.7852 [0.7798, 0.7910]
Observations	10,080,000	8,480,000

526 Excluding the 1,600,000 pre-onset observations, 15.87% of the partition,
527 raised macro-F1 by 0.0854, accuracy by 0.1195 and MCC by 0.1231. No
528 model was refitted and no hyperparameter was changed.

529 Two diagnostics establish that this is a labeling effect rather than a
530 favourable choice of subset. First, the frozen model assigned 16.10% of the
531 pre-onset observations to class 0 and 75.86% to classes 3, 9 or 15, that is,
532 92.0% to the group of conditions that are indistinguishable from nominal op-
533 eration, and only 4.20% to the fault label they nominally carry. The model
534 therefore identifies the pre-onset segment as a period in which nothing de-
535 tectable is occurring, which is correct, and literal labeling scores this as error.
536 Second, restricting the evaluation to post-onset observations raised the recall
537 of the 17 well-separated classes to between 0.8015 and 0.9999, the upper value
538 corresponding to essentially complete recovery of faults 6 and 7, while the
539 ambiguity group improved only marginally (F1 of 0.2217, 0.3238, 0.2100 and
540 0.2106 for classes 0, 3, 9 and 15, against 0.1718, 0.2357, 0.1635 and 0.1683
541 under literal labels). The ambiguity group is therefore not a labeling artifact;
542 the ceiling on the remaining seventeen classes is.

543 The two phenomena are nevertheless connected. Class 3 was predicted
544 1,237,562 times against a true support of 480,000, an excess of 757,562 ob-
545 servations. A substantial part of that excess consists of pre-onset segments
546 belonging to other faults, which the model routes to the weakly observable
547 conditions because, at those instants, nothing has yet happened. The over-
548 prediction of class 3 is thus partly a consequence of the labeling convention
549 and partly a consequence of genuine overlap, and only the post-onset analysis
550 separates the two.

551 This distinction matters for comparability. As documented in Section 2.5,
552 published observation-level results on the expanded TEP dataset do not con-

Table 8: Effect of post-onset labeling across model families. The last column reports the relative accuracy gain $\Delta A/A_P$, whose structural upper bound is $1 - 8,480,000/10,080,000 = 0.1587$.

Model	Macro-F1 (L)	Macro-F1 (P)	$\Delta A/A_P$
Decision tree	0.6330	0.7167	0.148
Extra trees	0.6965	0.7786	0.148
Random forest	0.7081	0.7912	0.149
HistGradientBoosting	0.7128	0.7972	0.149
XGBoost [†]	0.7200	0.8054	0.150
Logistic regression	0.4283	0.4698	0.142
Linear SVM	0.4207	0.4655	0.142

[†]Frozen model of Table 4, complete test partition.

553 sistently state whether pre-onset samples were retained, discarded, or rela-
554 beled, yet the three choices are separated by several points of macro-F1 for an
555 identical model. Two studies reporting different scores may therefore differ
556 in labeling convention rather than in method. We recommend that the treat-
557 ment of the onset window be reported explicitly as part of the experimental
558 protocol, in the same way that the unit of partitioning is reported.

559 4.6. Generality of the labeling effect across model families

560 If the effect is a property of the labeling convention rather than of the
561 selected classifier, it should appear in every model evaluated on the same
562 partition. An auxiliary experiment was therefore rescored under both pro-
563 tocols. It predates the frozen benchmark, uses a per-class subsample of the
564 test partition and includes an extremely randomized trees model in place
565 of XGBoost, so only the differences between protocols are interpreted. No
566 model was refitted.

567 The five tree-based models gained between 0.0821 and 0.0854 in macro-
568 F1, a spread of 0.003 across estimators whose capacities differ by orders of
569 magnitude, from a single unpruned tree to a regularized boosting ensemble.
570 No overfitting mechanism produces that uniformity, because the degradation
571 associated with overfitting scales with model capacity.

572 The relative accuracy gain explains the pattern. If a classifier assigns
573 pre-onset observations to any class other than the labeled fault, its literal
574 accuracy is its post-onset accuracy scaled by the retained fraction of the par-
575 tition, so $\Delta A/A_P$ is bounded above by 0.1587 independently of the model.

576 The seven models fall between 0.142 and 0.150, the residual being the small
577 fraction of pre-onset observations that match the literal label by chance. The
578 linear baselines gain less in absolute terms, 0.0415 and 0.0448, only because
579 they start lower, and they route far fewer pre-onset observations to the group
580 $\{0, 3, 9, 15\}$, 20.6% and 41.9% against 65.7–70.2% for the ensembles, yet are
581 penalized in the same proportion. Any observation-level result reported on
582 this dataset under literal labels therefore understates the post-onset discrim-
583 inative capacity of the model by a factor predictable from its own accuracy.

584 4.7. Structure of the remaining errors

585 The normalized confusion matrix in Fig. 1 shows that errors were not
586 uniformly distributed. They concentrated in the four-class group $\{0, 3, 9, 15\}$.
587 Among true class-9 observations, 32.39% were predicted as class 3, 20.24%
588 as class 0, and 18.08% as class 15. Normal observations were assigned to
589 classes 3, 9, and 15 in 32.02%, 21.51%, and 18.24% of cases, respectively. For
590 class 15, the corresponding proportions were 30.96%, 20.53%, and 20.03%
591 for predicted classes 3, 9, and 0.

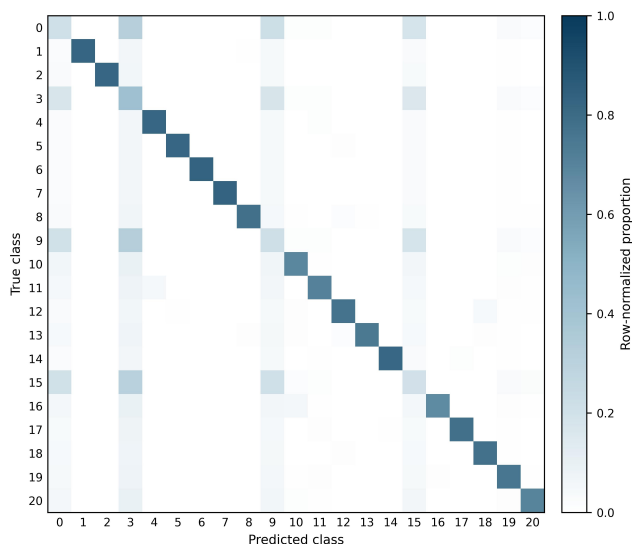


Figure 1: Row-normalized confusion matrix for the frozen XGBoost model on the official test partition.

592 Class 3 was predicted 1,237,562 times despite having 480,000 true obser-
593 vations. This excess explains its low precision and demonstrates that the

Table 9: Performance and computational cost of the six families under repeated validation (mean over five seeds, 420,000 training observations per repetition). Times measured on the environment of Section 3.7.

Model	Macro-F1	MCC	Train (s)	Inference (μ s/obs)	Size (MB)
XGBoost	0.7683	0.7426	864.7	7.03	18.9
HistGradientBoosting	0.7643	0.7391	73.5	29.40	16.9
Random Forest	0.7398	0.7159	477.3	20.29	8703.8
Decision Tree	0.6714	0.6537	54.0	0.24	28.8
Logistic Regression	0.4685	0.4567	65.4	0.15	0.005
Linear SVM	0.4438	0.4377	4.1	0.16	0.006

model used class 3 as a frequent destination for ambiguous trajectories. The persistence of the same difficult group during repeated validation and official testing suggests an intrinsic overlap under an observation-level representation rather than an isolated sampling artifact.

4.8. Computational performance

Table 9 reports the cost of all six families under identical conditions during repeated validation, which is required to answer the performance–cost question and cannot be inferred from the selected model alone. Training times are reported as means over the five seeds and are not directly comparable with the 170.34 s of Table 4, which was measured during the final evaluation on twice as many observations. The degree of parallelism was an invocation-time argument of each stage rather than a fixed setting, and it was not recorded in the run manifests, so the absolute times of the two stages reflect different thread counts on a 24-core host. The comparison between families within Table 9 is unaffected, because those six runs shared a single invocation, but the cross-stage comparison is not meaningful and no conclusion is drawn from it. Recording the degree of parallelism alongside the timing is a reproducibility requirement that this benchmark did not initially meet, and it has been added to the released code.

Final XGBoost training required 170.34 seconds. Prediction over 10.08 million observations required 56.57 seconds, corresponding to 0.005612 ms per observation or approximately 178,000 observations per second. The serialized final model occupied approximately 6.8 MB. These values indicate that the selected classifier is computationally feasible for high-throughput offline analysis and potentially for online scoring, although end-to-end deployment

latency must also include data acquisition, preprocessing, communication, and decision logic.

5. Discussion

5.1. Nonlinearity and model ranking

The ranking provides strong empirical evidence that the TEP classification boundary is not adequately represented by the evaluated linear models. XGBoost, HistGradientBoosting, and random forest substantially outperformed logistic regression and linear SVM even though all models received the same standardized predictors and run-level partitions. Tree ensembles can represent threshold effects and high-order interactions without requiring an explicit feature expansion, which is consistent with the nonlinear and feedback-driven nature of the TEP.

XGBoost and HistGradientBoosting differed by 0.0040 in mean macro-F1 with overlapping intervals, so the defensible conclusion is that XGBoost won under the prespecified rule, not that it is superior under all resampling regimes. Table 9 shows that the practical choice does not favour either: HistGradientBoosting trained 11.8 times faster, while XGBoost inferred 4.2 times faster, so the decision follows from whether the deployment is retraining-bound or scoring-bound. Model size is a third and largely independent constraint: the random forest required some 460 times the XGBoost artifact for 0.0285 less macro-F1, which rules it out for embedded deployment without being visible in any predictive metric. None of these conclusions can be drawn when only the selected model is profiled.

5.2. Why global performance is insufficient

The official macro-F1 of 0.7200 summarizes useful performance, but it does not describe a uniformly reliable 21-class diagnostic system. If only macro-F1 or accuracy had been reported, the strong results for 17 classes would obscure the near-failure of normal operation and faults 3, 9, and 15. In operational terms, low precision for normal operation and extensive exchange between normal and faulty labels can increase false alarms and missed interventions.

This result illustrates why class-level precision, recall, F1, and confusion structure should accompany aggregate measures in industrial diagnosis. It also supports a distinction between *benchmark discrimination* and *deployment readiness*. The present model is a strong benchmark classifier for most

654 faults, but additional safeguards are necessary before it could support au-
655 tonomous decisions.

656 *5.3. Implications for method selection in industrial practice*

657 Benchmark figures circulate as evidence of capability: a method reporting
658 a macro-F1 of 0.80 on the TEP appears preferable to one reporting 0.72.
659 Sections 4.5 and 4.6 show that a difference of exactly that size separates two
660 labeling conventions applied to an identical model. The question to put to a
661 reported figure is therefore not only which algorithm produced it, but over
662 which observations it was computed.

663 The fixed injection instant of the benchmark has no counterpart in an
664 operating plant, but the interval it stands for does. A disturbance entering
665 a unit does not become visible in the instrumentation at the moment it
666 occurs. It propagates through transport lag, and it is actively opposed by the
667 control loops, whose rejection of the disturbance is precisely what conceals it:
668 the difficulty of detecting faults inside chemical-process control loops due to
669 feedback masking is long recognized [20]. For incipient conditions the margin
670 is narrow to begin with, with reported amplitudes on the order of one to ten
671 per cent of the fault-free signal [21], so the deviation may remain within
672 normal variability for an extended period. The interval between occurrence
673 and annunciation is itself a design quantity in alarm management, where
674 deadbands and delay timers trade detection delay against nuisance alarms
675 [22]. Whatever label a historical dataset attaches to observations in that
676 interval, a model receives data carrying no signature of the condition it is
677 expected to report, and a figure of merit computed over them measures the
678 labeling convention rather than the diagnostic system.

679 The group $\{0, 3, 9, 15\}$ makes the same point in a sharper form. These
680 conditions survive the correction of Section 4.5 because they leave no persis-
681 tent signature in the 52 measured variables. For a deployment this is not a
682 modeling deficiency to be repaired with a better classifier; it is a statement
683 about what the installed instrumentation can observe. The literature treats
684 this as a design problem rather than an algorithmic one: sensor location can
685 be posed explicitly against fault-diagnostic observability criteria [23], and a
686 comprehensive formulation combining diagnosability, reliability and sensor
687 cost has been demonstrated on the Tennessee Eastman flowsheet itself [24].
688 The implication for practice is that when a condition proves undiagnosable
689 from the available measurements, the productive response is to revisit the
690 measurement set, not to continue tuning the classifier.

691 The cost results of Table 9 show that no model dominates. HistGradient-
692 Boosting trained 11.8 times faster than XGBoost for a difference of 0.0040
693 in macro-F1, while XGBoost inferred 4.2 times faster; the random forest re-
694 quired approximately 8.7 GB. Which constraint binds is a property of the
695 installation rather than of the benchmark: a model retrained weekly on a
696 workstation and a model scoring within the scan cycle of a controller are
697 limited by different quantities, and neither limit is visible in a predictive
698 metric.

699 The common thread is that benchmark performance and deployment
700 readiness are distinct properties, and that the distance between them is not
701 only a matter of model quality. It also depends on how the evaluation was
702 constructed, on what the instrumentation can observe, and on the resources
703 available where the model will run.

704 *5.4. Interpretation of the ambiguity group*

705 The identity of the difficult group is not a new finding, and it would be
706 misleading to present it as one. Faults 3, 9 and 15 have been reported as
707 weakly detectable since the earliest systematic studies of the TEP: moni-
708 toring statistics based on PCA, discriminant PLS and Fisher discriminant
709 analysis report missed-detection rates close to those of nominal operation
710 for exactly these conditions [5, 4], and later comparative studies reproduce
711 the pattern across a wide range of data-driven statistics [18]. The contribu-
712 tion here is the scale and the protocol: the same group is recovered under
713 run-level partitioning, on 10.08 million independent test observations, and it
714 persists after the labeling artifact of Section 4.5 is removed. This strengthens
715 the interpretation that the overlap is a property of the measured variables
716 rather than of any particular estimator.

717 The mechanism, however, constrains which remedies can work. If these
718 disturbances leave no persistent signature in the 52 measured variables, then
719 temporal aggregation over a run reduces the variance of an estimate that is
720 already centered on an ambiguous region: it will sharpen the decision without
721 moving it toward the correct class. Run-level aggregation should therefore
722 be expected to help the 17 separable classes, whose pre-onset transients are
723 transient, far more than the group $\{0, 3, 9, 15\}$, and this asymmetry is itself a
724 testable prediction. A hierarchical design that isolates the ambiguous group
725 and applies a discriminator with explicit memory to classes 0, 3, 9 and 15
726 is the more promising direction, but its plausibility rests on whether any
727 signature exists at all; establishing this is a question of observability, not

728 of classifier capacity, and it is better addressed with residual-based or first-
729 principles analysis than with additional supervised tuning.

730 Both proposals arise from test-set diagnosis and must therefore be eval-
731 uated under a newly defined development/test protocol; the present official
732 test must not be reused for tuning them.

733 5.5. *Threats to validity*

734 The TEP is a high-fidelity simulation, not a physical plant, so the results
735 need not transfer to sensor drift, maintenance changes, unmodeled distur-
736 bances or evolving operating modes; and the evaluation is pointwise mul-
737 ticlass diagnosis, which does not measure detection delay, false alarms per
738 run or sustained alarm logic. The uncertainty estimates rest on five seeds
739 and characterize the implemented experiment rather than all possible parti-
740 tions, and the hyperparameter search was deliberately limited to three can-
741 didates per family, so individual algorithms, particularly the linear baselines,
742 might improve under a wider search. The linear SVM in particular was a
743 stochastic-gradient surrogate and should not be read as the best attainable
744 margin classifier.

745 Three further caveats bear on quantities reported here. The final model
746 used 840,000 of the 5,250,000 available development observations and selec-
747 tion was performed at 420,000; Section 4.2 bounds the residual headroom at
748 0.0086 in post-onset macro-F1 under quadrupling, so the reported figures are
749 conservative, but a ranking established at one scale need not be invariant to
750 it. The degree of parallelism was not recorded in the run manifests, so the
751 times in Table 9 support ordinal comparison between families, which shared
752 one invocation, but not comparison against other stages. Finally, class sup-
753 port is balanced by construction, whereas industrial faults are rare; metrics,
754 thresholds and alarm costs would require recalibration under field prevalence.
755 Interpretation throughout focused on output behaviour and did not include
756 feature attribution or causal analysis.

757 6. Conclusion

758 This work presented a leakage-controlled benchmark of six classical clas-
759 sifiers for 21-class fault diagnosis in the Tennessee Eastman Process, using
760 run-level splitting, train-only standardization, separate tuning and valida-
761 tion, five-seed repetition and a single frozen test evaluation.

762 XGBoost ranked first in repeated validation (0.7683 ± 0.0017) and pro-
763 duced a test macro-F1 of 0.7200, accuracy of 0.6754 and MCC of 0.6621 over
764 10.08 million observations under literal labeling, rising to 0.8054, 0.7949 and
765 0.7852 once the pre-onset segment is excluded, with no refitting. Rescoring
766 the validation partition under the same protocol turns an apparent general-
767 ization gap of +0.0483 into -0.0125 . Several of the seventeen well-diagnosed
768 classes were shown to sit at the structural recall ceiling imposed by the la-
769 beling convention rather than at the limit of the classifier, while normal
770 operation and faults 3, 9 and 15 remain confounded after the artifact is re-
771 moved.

772 Three conclusions follow. Nonlinear tree ensembles diagnose most TEP
773 conditions effectively. Aggregate scores overstate practical reliability when
774 a small group of classes dominates the remaining errors. And a benchmark
775 result on these data is not comparable across studies unless the treatment
776 of the fault-onset window is stated: a single frozen model differs by 0.085 in
777 macro-F1 between two defensible conventions, more than most improvements
778 reported between competing architectures, and the effect is common to all
779 seven model families tested. We recommend that the treatment of the onset
780 window be reported as part of the experimental protocol, alongside the unit
781 of partitioning.

782 Future work should investigate temporal aggregation, hierarchical diag-
783 nosis, probability calibration and cost-sensitive alarm rules under a new
784 untouched test protocol. Sequence models raise the question in a richer
785 form: windowed labeling introduces a third category of observations, those
786 spanning the injection instant, whose proportion depends on the windowing
787 scheme of each study.

788 Data and Code Availability

789 The expanded Tennessee Eastman simulation data are publicly avail-
790 able from Harvard Dataverse [7]. The CSV conversion used in this study
791 is available through Kaggle [15]. All code, configuration files, run-level split
792 manifests, frozen hyperparameters, five-seed validation outputs, final pre-
793 dictions, class-level metrics, confusion matrices, SHA-256 data hashes, and
794 the environment specification are available for review at **[TBD: anony-**
795 **mous.4open.science URL]**. The non-anonymized repository and its archival
796 DOI are cited in the accepted version. The analysis of the fault-onset win-
797 dow can be reproduced from the deposited predictions without refitting any

798 model.

799 CRediT Authorship Contribution Statement

800 **First author:** Conceptualization, Investigation, Validation, Writing –
801 original draft, Writing – review & editing. **Second author:** Investigation,
802 Validation, Writing – original draft, Writing – review & editing. **Third**
803 **author:** Conceptualization, Methodology, Software, Formal analysis, Data
804 curation, Writing – original draft, Visualization, Supervision, Project admin-
805 istration.

806 Declaration of Competing Interest

807 The authors declare that they have no known competing financial inter-
808 ests or personal relationships that could have appeared to influence the work
809 reported in this paper.

810 References

- 811 [1] V. Venkatasubramanian, R. Rengaswamy, K. Yin, and S. N. Kavuri,
812 “A review of process fault detection and diagnosis: Part I: Quantitative
813 model-based methods,” *Computers & Chemical Engineering*, vol. 27, no.
814 3, pp. 293–311, 2003. doi: 10.1016/S0098-1354(02)00160-6.
- 815 [2] S. Yin, S. X. Ding, X. Xie, and H. Luo, “A review on basic data-
816 driven approaches for industrial process monitoring,” *IEEE Transac-*
817 *tions on Industrial Electronics*, vol. 61, no. 11, pp. 6418–6428, 2014. doi:
818 10.1109/TIE.2014.2301773.
- 819 [3] C. Ji and W. Sun, “A review on data-driven process monitoring methods:
820 Characterization and mining of industrial data,” *Processes*, vol. 10, no.
821 2, art. 335, 2022. doi: 10.3390/pr10020335.
- 822 [4] L. H. Chiang, E. L. Russell, and R. D. Braatz, *Fault Detection and*
823 *Diagnosis in Industrial Systems*. London, UK: Springer, 2001. doi:
824 10.1007/978-1-4471-0347-9.
- 825 [5] L. H. Chiang, E. L. Russell, and R. D. Braatz, “Fault diagnosis in chem-
826 ical processes using Fisher discriminant analysis, discriminant partial
827 least squares, and principal component analysis,” *Chemometrics and*
828 *Intelligent Laboratory Systems*, vol. 50, no. 2, pp. 243–252, 2000. doi:
829 10.1016/S0169-7439(99)00061-1.

- 830 [6] J. J. Downs and E. F. Vogel, “A plant-wide industrial process control
831 problem,” *Computers & Chemical Engineering*, vol. 17, no. 3, pp. 245–
832 255, 1993. doi: 10.1016/0098-1354(93)80018-I.
- 833 [7] C. A. Rieth, B. D. Amsel, R. Tran, and M. B. Cook, “Additional Ten-
834 nessee Eastman Process simulation data for anomaly detection evalua-
835 tion,” Harvard Dataverse, V1, 2017. doi: 10.7910/DVN/6C3JR1.
- 836 [8] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification
837 and Regression Trees*. Belmont, CA, USA: Wadsworth, 1984.
- 838 [9] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, pp. 5–32,
839 2001. doi: 10.1023/A:1010933404324.
- 840 [10] J. H. Friedman, “Greedy function approximation: A gradient boosting
841 machine,” *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
842 doi: 10.1214/aos/1013203451.
- 843 [11] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting sys-
844 tem,” in *Proceedings of the 22nd ACM SIGKDD International Confer-
845 ence on Knowledge Discovery and Data Mining*, 2016, pp. 785–794. doi:
846 10.1145/2939672.2939785.
- 847 [12] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*,
848 vol. 20, pp. 273–297, 1995. doi: 10.1007/BF00994018.
- 849 [13] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal
850 of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- 851 [14] B. W. Matthews, “Comparison of the predicted and observed secondary
852 structure of T4 phage lysozyme,” *Biochimica et Biophysica Acta*, vol.
853 405, no. 2, pp. 442–451, 1975. doi: 10.1016/0005-2795(75)90109-9.
- 854 [15] A. Melo, “Tennessee Eastman Process CSV: Process simulation
855 data for anomaly detection evaluation,” Kaggle Dataset, accessed
856 Aug. 2026. [Online]. Available: [https://www.kaggle.com/datasets/
857 afnriomelo/tep-csv](https://www.kaggle.com/datasets/afnriomelo/tep-csv).
- 858 [16] C. A. Rieth, B. D. Amsel, R. Tran, and M. B. Cook, “Issues and advances
859 in anomaly detection evaluation for joint human-automated systems,” in
860 *Advances in Human Factors in Robots and Unmanned Systems* (AHFE
861 2017), Cham: Springer, 2018, pp. 52–63.
- 862 [17] C. Reinartz, M. Kulahci, and O. Ravn, “An extended Tennessee Eastman
863 simulation dataset for fault-detection and decision support systems,”
864 *Computers & Chemical Engineering*, vol. 149, art. 107281, 2021. doi:
865 10.1016/j.compchemeng.2021.107281.
- 866 [18] S. Yin, S. X. Ding, A. Haghani, H. Hao, and P. Zhang, “A compar-
867 ison study of basic data-driven fault diagnosis and process monitor-

- ing methods on the benchmark Tennessee Eastman process,” *Journal of Process Control*, vol. 22, no. 9, pp. 1567–1581, 2012. doi: 10.1016/j.jprocont.2012.06.009.
- [19] J. Demšar, “Statistical comparisons of classifiers over multiple data sets,” *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.
- [20] Y.-J. Park, S.-K. S. Fan, and C.-Y. Hsu, “A review on fault detection and process diagnostics in industrial processes,” *Processes*, vol. 8, no. 9, art. 1123, 2020. doi: 10.3390/pr8091123.
- [21] H. Safaeipour, M. Forouzanfar, and A. Casavola, “A survey and classification of incipient fault diagnosis approaches,” *Journal of Process Control*, vol. 97, art. 102422, 2021. doi: 10.1016/j.jprocont.2020.11.005.
- [22] N. A. Adnan, I. Izadi, and T. Chen, “On expected detection delays for alarm systems with deadbands and delay-timers,” *Journal of Process Control*, vol. 21, no. 9, pp. 1318–1331, 2011. doi: 10.1016/j.jprocont.2011.06.019.
- [23] R. Raghuraj, M. Bhushan, and R. Rengaswamy, “Locating sensors in complex chemical plants based on fault diagnostic observability criteria,” *AIChE Journal*, vol. 45, no. 2, pp. 310–322, 1999.
- [24] M. Bhushan and R. Rengaswamy, “Comprehensive design of a sensor network for chemical plants based on various diagnosability and reliability criteria. 2. Applications,” *Industrial & Engineering Chemistry Research*, vol. 41, no. 7, pp. 1840–1860, 2002.
- [25] A. Raich and A. Çinar, “Statistical process monitoring and disturbance diagnosis in multivariable continuous processes,” *AIChE Journal*, vol. 42, no. 4, pp. 995–1009, 1996.
- [26] P. R. Lyman and C. Georgakis, “Plant-wide control of the Tennessee Eastman problem,” *Computers & Chemical Engineering*, vol. 19, no. 3, pp. 321–331, 1995.
- [27] M. Misra, H. H. Yue, S. J. Qin, and C. Ling, “Multivariate process monitoring and fault diagnosis by multi-scale PCA,” *Computers & Chemical Engineering*, vol. 26, no. 9, pp. 1281–1293, 2002.
- [28] L. H. Chiang, M. E. Kotanchek, and A. K. Kordon, “Fault diagnosis based on Fisher discriminant analysis and support vector machines,” *Computers & Chemical Engineering*, vol. 28, no. 8, pp. 1389–1401, 2003.
- [29] A. Kulkarni, V. K. Jayaraman, and B. D. Kulkarni, “Knowledge incorporated support vector machines to detect faults in Tennessee Eastman process,” *Computers & Chemical Engineering*, vol. 29, no. 10, pp. 2128–2133, 2005.

- 906 [30] N. F. Thornhill and A. Horch, “Advances and new directions in plant-
907 wide disturbance detection and diagnosis,” *Control Engineering Prac-*
908 *tice*, vol. 15, no. 10, pp. 1196–1206, 2007.
- 909 [31] Y. W. Zhang, “Enhanced statistical analysis of nonlinear process using
910 KPCA, KICA and SVM,” *Chemical Engineering Science*, vol. 64, no. 5,
911 pp. 801–811, 2009.
- 912 [32] S. Yin, X. Gao, H. R. Karimi, and X. Zhu, “Study on support vec-
913 tor machine-based fault detection in Tennessee Eastman process,” *Ab-*
914 *stract and Applied Analysis*, vol. 2014, art. 836895, 8 pages, 2014. doi:
915 10.1155/2014/836895.
- 916 [33] W. Sun, A. R. C. Paiva, P. Xu, A. Sundaram, and R. D. Braatz, “Fault
917 detection and identification using Bayesian recurrent neural networks,”
918 *Computers & Chemical Engineering*, vol. 141, art. 106991, 2020. doi:
919 10.1016/j.compchemeng.2020.106991.
- 920 [34] S. Kapoor and A. Narayanan, “Leakage and the reproducibility crisis
921 in machine-learning-based science,” *Patterns*, vol. 4, no. 9, art. 100804,
922 2023. doi: 10.1016/j.patter.2023.100804.
- 923 [35] I. Lomov, M. Lyubimov, I. Makarov, and L. E. Zhukov, “Fault detection
924 in Tennessee Eastman process with temporal deep learning models,”
925 *Journal of Industrial Information Integration*, vol. 23, art. 100216, 2021.
926 doi: 10.1016/j.jii.2021.100216.
- 927 [36] G. S. Chadha and A. Schwung, “Comparison of deep neural network ar-
928 chitectures for fault detection in Tennessee Eastman process,” in *Proc.*
929 *22nd IEEE International Conference on Emerging Technologies and*
930 *Factory Automation (ETFA)*, 2017, pp. 1–8.
- 931 [37] P. Agarwal et al., “Hierarchical deep LSTM for fault detection and diag-
932 nosis for a chemical process,” *Processes*, vol. 10, no. 12, art. 2557, 2022.
933 doi: 10.3390/pr10122557.
- 934 [38] C. Anitha et al., “Fault diagnosis of Tennessee Eastman process with
935 detection quality using IMVOA with hybrid DL technique in IIoT,” *SN*
936 *Computer Science*, vol. 4, no. 5, art. 458, 2023. doi: 10.1007/s42979-
937 023-01851-9.
- 938 [39] X. Gao and J. Hou, “An improved SVM integrated GS-PCA fault di-
939 agnosis approach of Tennessee Eastman process,” *Neurocomputing*, vol.
940 174, pp. 906–911, 2016.
- 941 [40] W. Ku, R. H. Storer, and C. Georgakis, “Disturbance detection and
942 isolation by dynamic principal component analysis,” *Chemometrics and*
943 *Intelligent Laboratory Systems*, vol. 30, no. 1, pp. 179–196, 1995.

- 944 [41] E. L. Russell, L. H. Chiang, and R. D. Braatz, "Fault detection in in-
945 dustrial processes using canonical variate analysis and dynamic principal
946 component analysis," *Chemometrics and Intelligent Laboratory Systems*,
947 vol. 51, no. 1, pp. 81–93, 2000.
- 948 [42] P. P. Odiowei and Y. Cao, "Nonlinear dynamic process monitoring using
949 canonical variate analysis and kernel density estimations," *IEEE Trans-*
950 *actions on Industrial Informatics*, vol. 6, no. 1, pp. 36–45, 2010.